

A tener en cuenta

- En lo posible, usar métodos de array.
- Las funciones no deben modificar los objetos que se les pasan por argumentos.
- Las entradas proporcionadas han de ser comprobadas.

Empleados

RA 4	Programa código para clientes web analizando y utilizando estructuras definidas por el usuario.	Puntuación	% RA
		10	100%
CEv	a) Se han clasificado y utilizado las funciones predefinidas del lenguaje. b) Se han creado y utilizado funciones definidas por el usuario. c) Se han reconocido las características del lenguaje relativas a la creación y uso de matrices (arrays). d) Se han creado y utilizado matrices (arrays). e) Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones. k) Se ha depurado y documentado el código.		

Se cuenta con la estructura de datos **empleados** que contiene una lista de objetos, donde cada objeto representa un empleado con diferentes atributos. Cada empleado tiene las siguientes propiedades:

- **nombre (string)**: Nombre completo del empleado.
- **pais (string)**: País de residencia del empleado.
- **edad (number)**: Edad del empleado.
- **area (string)**: Área en la que trabaja el empleado (ej. "QA", "Desarrollo", "Recursos Humanos").
- **puesto (string)**: Puesto específico dentro del área (ej. "Backend Developer", "Frontend Developer").
- **seniority (string)**: Nivel de experiencia del empleado. Puede ser: "Trainee", "Junior", "Semisenior", "Senior".
- **suelo (number)**: Salario en una unidad monetaria determinada.
- **haceGuardia (boolean)**: Indica si el empleado está disponible para realizar guardias (turnos fuera del horario laboral).
- **lenguajes (array de strings)**: Lista de lenguajes de programación que domina el empleado.

Realiza sobre el array de objetos **empleados** contenido en **data-empleados.js**, las siguientes funciones:

1. **empleadosDeGuardiaPorPais**: Devuelve un objeto donde las claves son los nombres de los países y los valores son arrays con los empleados de cada país que hacen guardia.
(1,5 puntos)

Salida:

```
Object { Colombia: (9) [...], Bolivia: (4) [...], Venezuela: (5) [...], Paraguay: (5) [...], "México": (7) [...], Brasil: (5) [...], Uruguay: (2) [...], Chile: (2) [...], "Perú": (5) [...], Argentina: (1) [...] }
  Argentina: Array [ {...} ]
    0: Object { nombre: "Mercedes Roman", pais: "Argentina", edad: 48, ... }
      length: 1
  ...
```

2. **topNEmpleadosConMayorSueldo**: Toma como parámetro un número N y devuelve un array con los N empleados mejor pagados, ordenados de mayor a menor sueldo.
(1 punto)

Entrada: 5

Salida:

```
Array(5) [ {...}, {...}, {...}, {...}, {...} ]
  0: Object { nombre: "Short Butler", pais: "Venezuela", edad: 38, ... }
  1: Object { nombre: "Mercedes Roman", pais: "Argentina", edad: 48, ... }
  2: Object { nombre: "Kaufman Vasquez", pais: "Argentina", edad: 37, ... }
  3: Object { nombre: "Tanisha Hendrix", pais: "Chile", edad: 34, ... }
  4: Object { nombre: "Burton Parsons", pais: "Argentina", edad: 42, ... }
      length: 5
```

3. **empleadosConLenguajeEspecifico**: Toma dos parámetros:
 - Un número que representa el mínimo de lenguajes que deben conocer.
 - Un string que representa un lenguaje requerido.Devuelve un array con empleados que cumplan ambas condiciones.

(1 punto)

Entrada:

Ingrese el mínimo de lenguajes: 4

Ingrese el lenguaje requerido: *Java*

Salida:

```
Array(32) [ {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...},  
{...}, ... ]  
0: Object { nombre: "Eunice Browning", pais: "Colombia",  
edad: 20, ... }  
1: Object { nombre: "Floyd Reyes", pais: "Paraguay",  
edad: 29, ... }  
2: Object { nombre: "Downs Mccray", pais: "Brasil",  
edad: 46, ... }  
3: Object { nombre: "Madelyn Berger", pais: "Colombia",  
edad: 34, ... }  
...  
31: Object { nombre: "Cummings Woods", pais:  
"Venezuela", edad: 31, ... }  
length: 32
```

4. **empleadosQuePuedenAscender**: Devuelve un array con los empleados que no son "Senior" y que, si se les subiera de categoría, no superarían los 100,000 en sueldo base.

(1 punto)

Salida:

```
Array(69) [ {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}, {...},  
{...}, ... ]  
0: Object { nombre: "Wade Morgan", pais: "México", edad:  
20, ... }  
1: Object { nombre: "Eunice Browning", pais: "Colombia",  
edad: 20, ... }  
2: Object { nombre: "Floyd Reyes", pais: "Paraguay",  
edad: 29, ... }  
...  
68: Object { nombre: "Lakeisha Martin", pais:  
"Venezuela", edad: 44, ... }  
length: 69
```

5. **empleadoMasJovenConMayorSueldo**: Devuelve el empleado más joven de entre los que tienen el mayor sueldo registrado.

(1,5 puntos)

Salida:

```
Object { nombre: "Short Butler", pais: "Venezuela", edad:  
38, area: "Seguridad", puesto: "DevOps", seniority:  
"Junior", sueldo: 99270, haceGuardia: true, lenguajes: [] }
```

6. **porcentajeEmpleadosPorSeniority**: Devuelve un objeto donde cada clave es un nivel de seniority y el valor es el porcentaje de empleados en ese nivel respecto al total.

(1,25 puntos)

Salida:

```
Object { Semisenior: 18, Junior: 28, Trainee: 28, Senior: 26 }
```

7. **calculoCostosEmpresa**: Devuelve un objeto con: totalSueldosBrutos, totalObraSocial (3% *del sueldo*), totalJubilacion (11% *del sueldo*), totalSueldosNetos (*sueldo – (ObraSocial – Jubilación)*)

(1,75 puntos)

Salida:

```
Object { totalSueldosBrutos: 5548285, totalObraSocial: 166448.55, totalJubilacion: 610311.3499999999, totalSueldosNetos: 4771525.099999999 }
```

8. Implementa un **menú interactivo** en la consola para realizar las llamadas a las funciones en index.js. **Organiza y documenta** el código.

(1 punto)